



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

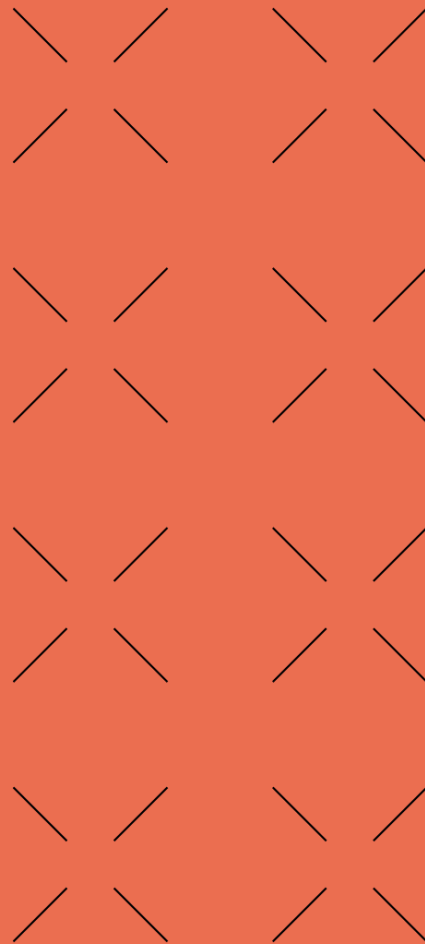
ceedcv CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 04 — Estructures de control i excepcions

Bloc 01

1r CFGS DAW · Programació

Sergi García Barea



if: el semàfor del codi

La idea en una frase

“

`if` és el semàfor del codi: si la condició és `true`, deixa passar al bloc; si és `false`, el redirigeix al `else` (o continua com si res).

En unitats anteriors els teus programes jutjaven amb operadors relacionals i ternaris, però eixa justícia durava una línia. Ara arriba la justícia de debò: **blocs sencers** de codi que s'executen o no segons el que decidisca Java.

if i else: la estructura bàsica

Semafor.java

Java

```
int edat = 17;
if (edat ≥ 18) {
    System.out.println("Pots votar.");
} else {
    System.out.println("Encara no pots votar.");
}
```



El `else` és **opcional**: un `if` sol és perfectament legal. Si la condició falla, Java continua com si res.

else if: quan hi ha més de dos camins

Notes.java

Java

```
int nota = 7;
if (nota ≥ 9) {
    System.out.println("Excel·lent");
} else if (nota ≥ 7) {
    System.out.println("Notable");
} else if (nota ≥ 5) {
    System.out.println("Aprovat");
} else {
    System.out.println("Suspés");
}
```



Java avalua les condicions **en ordre, de dalt a baix**: quan una dona `true`, executa el seu bloc i **es salta la resta**. L'ordre importa: de la més **exigent** a la més **permissiva**.

L'ordre està invertit: el semàfor confús

✗ Aquest programa imprimeix...

```
if (nota ≥ 5) → "Aprovat"  
else if (nota ≥ 7) → "Notable"  
else if (nota ≥ 9) → "Excel·lent"
```

Amb nota = 8 imprimeix "**Aprovat**": el primer if que es compleix guanya, encara que 8 també compliria 7 i 9.

✓ La correcció: ordena al revés

```
if (nota ≥ 9) primer,  
else if (nota ≥ 7) després,  
else if (nota ≥ 5) al final.
```

De la més exigent a la més permissiva: eixe és el 90% dels bugs d'esta unitat.

If anidats: semàfors dins de semàfors

```
Conduir.java

if (edat ≥ 18) {
    if (teCarnet) {
        System.out.println("Pots conduir.");
    } else {
        System.out.println("Et falta el carnet.");
    }
} else {
    System.out.println("Encara no pots conduir.");
}
```

Un if pot viure dins d'un altre: útil per a decidir *dins* d'una decisió.



No convertesques els teus programes en les **Torres Kio**: més de **3 nivells** d'anidament és senyal que estàs fent les coses estrany.

El ternari: el if de butxaca

Ternari.java

Java

```
// ✅ Bé: ternari per a triar valor
double preuFinal = (dia.equals("divendres"))
    ? preu * 0.9
    : preu;

// ✅ Bé: if quan hi ha varies línies
if (saldo < 0) {
    System.out.println("Números rojos.");
    System.out.println("Revisa despeses.");
} else {
    System.out.println("Saldo sa.");
}
```

?

condicio ? valor1 : valor2 — un if-else en una línia, per a **assignar un valor**



Regla d'or: ternari per a assignar en una línia; if/else quan el bloc és llarg o fa més que assignar.

Exemple guiat: la discoteca municipal

Discoteca.java

Java

```
int edat = 16;
boolean acompanyat = true;
if (edat ≥ 18) {
    System.out.println("Entra, major d'edat.");
} else if (edat ≥ 16 && acompanyat) {
    System.out.println("Entra, però amb el teu acompanyant.");
} else {
    System.out.println("Ho sent, torna en uns quants anys.");
}
```

```
$ java Discoteca
```

```
Entra, però amb el teu acompanyant.
```

Mini-chequeig: if, else if, ternari



Respon en 30 segons:

1. Què fa Java si un if és false i no hi ha else?
2. En quin ordre has d'encadenar les condicions d'un else if?
3. Quan prefereixes un ternari a un if/else?
4. Què imprimix `String s = 5 > 3 ? "A" : "B";` ?



Respostes:

- Segueix executant la següent línia: l'if s'ignora en silenci
- De la més **exigent** a la més **permissiva**: el primer true es queda amb la decisió
- Quan només vols **assignar un valor** en una línia i les dues branques són curtes
- **"A"** — perquè `5 > 3` és true

switch: el menú del restaurant

La idea en una frase

“

`switch` és la carta d'un restaurant: mires el valor d'una variable i executes el `case` que coincideixi, sense encadenar vint `if`.

Quan has de triar entre moltes opcions amb un sol valor (dia de la setmana, talla, menú), una cadena d'`else if` funciona però és lletja. `switch` existeix per a això.

switch: les peces del puzzle

```
Dies.java
Java

int dia = 3;
switch (dia) {
    case 1:
        System.out.println("Dilluns");
        break;
    case 2:
        System.out.println("Dimarts");
        break;
    case 3:
        System.out.println("Dimecres");
        break;
    default:
        System.out.println("Desconegut");
        break;
}
```

- 1 `switch (variable)`: admet int, char, enum i String (Java 7+)
- 2 `case valor:` cada opció concreta a comparar
- 3 `break;` "fins ací he arribat": talla el switch
- ? `default:` el comodí "cap dels anteriors" (opcional)

El fall-through: error o superpoder?



Fall-through ACCIDENTAL

```
switch (plat) {  
  case 1:  
    System.out.println("Amanida");  
    // sense break: es cau  
  case 2:  
    System.out.println("Sopa");  
    break;  
}
```

Imprimeix **els dos plats**: un break oblidat converteix el switch en un tobogan.



Fall-through INTENCIONAL

```
switch (lletra) {  
  case 'a': case 'e': case 'i':  
  case 'o': case 'u':  
    System.out.println("Vocal");  
    break;  
}
```

Per a agrupar casos: si és qualsevol vocal, executa el bloc compartit. Elegant i compacte.

switch vs else if: el duel

Situació	Millor opció
Un valor i moltes opcions concretes (1...7, "S"/"M"/"L")	switch
Rangs o comparacions (≥ 18 , entre 10 i 20)	if/else if
Combinar diverses variables	if/else if
Comprovar null	if



Nota de futur: en Java 14+ existix el switch amb fletxes (`→`) que no necessita break i retorna valors. Ací aprenem el clàssic, que és el dels exàmens.

Exemple guiat: el menú del dia

MenuDia.java

Java

```
switch (dia) {  
    case "dilluns":  
        System.out.println("Llenties");  
        break;  
    case "dimarts":  
        System.out.println("Paella");  
        break;  
    case "dimecres":  
        System.out.println("Macarrons");  
        break;  
    default:  
        System.out.println("Cap de setmana");  
        break;  
}
```

```
$ java MenuDia // dia = "dimecres"  
Macarrons
```



Don Tip: quan veges un switch, compta els `break`: n'hi ha d'haver un per cada case no compartit. Si en falta algun, el teu programa es converteix en un tobogan.

Mini-chequeig: switch



Respon sense mirar:

1. Què passa si un case no té break?
2. Què fa el default?
3. Quan prefereixes switch a una cadena d'else if?
4. Com agrupes diversos valors en el mateix bloc?



Respostes:

- Es produïx el **fall-through**: el codi segueix amb els case següents fins a trobar un break
- És el comodí: s'executa si **cap** case coincideix. És opcional
- Quan compares **un valor amb moltes opcions concretes** (nombres, char, String)
- Escrivint els case seguits **sense break** entre ells i un sol bloc al final

Bucles: while i do-while

La idea en una frase

“

Un bucle és una cinta de córrer: executa el mateix bloc una vegada i una altra **mentre la condició siga true**.

T'imagines escriure "imprimeix de l'1 al 100" amb cent println? Copiar i enganxar és pecat. Els bucles fan la faena bruta per tu: repeteixen un bloc fins que els dius prou.

while: comprova i després corre

Intents.java

Java

```
int intents = 3;
while (intents > 0) {
    System.out.println("Queden " + intents);
    intents = intents - 1;
}
```

Queden 3

Queden 2

Queden 1



El while primer **mira la condició** i, si és true, executa el bloc. Si és false des del principi... **no executa res**.

El bucle infinit (i com eixir-ne)

Sentinel.java

Java

```
while (true) {  
    System.out.println("Socors");  
}  
// Ús legítim: sentinella  
String resposta = "";  
while (!resposta.equals("eixir")) {  
    System.out.print("Digues: ");  
    resposta = sc.nextLine();  
}
```



Si oblidess la línia que modifica la condició, el programa **no acaba mai**: benvingut al bucle infinit, el cotxe sense frens de la programació. En l'IDE, el botó de parar (■) és el teu millor amic.



Per què existix `while (true)`?
Perquè a vegades vols un bucle "per sempre" que es trenque **des de dins** amb `break` (punt 5). Un clàssic: llegir fins que l'usuari escriu "eixir" (el **sentinella**).

do-while: corre i després comprova

Menu.java

Java

```
int opcio;  
do {  
    System.out.println("1. Jugar  2. Eixir");  
    System.out.print("Tria: ");  
    opcio = sc.nextInt();  
} while (opcio != 1 && opcio != 2);  
System.out.println("Opció " + opcio);
```

La diferència amb while és l'**ordre**: el do-while executa el bloc **almenys una vegada** i comprova la condició al final.



El menú es mostra **sempre almenys una vegada** i es repeteix mentre l'usuari no trie 1 o 2: perfecte per a menús.

while vs do-while: el duel ràpid

Situació	Bucle ideal
No saps si tocarà executar el bloc	while
El bloc s'ha d'executar sí o sí una vegada	do-while
Llegir dades fins que l'usuari done el sentinella	while
Mostrar un menú fins que trie opció vàlida	do-while



És la diferència entre "**mira abans de creuar**" (while) i "**creua i després mira**" (do-while).

Exemple guiat: el compte arrere

● ● ● CompteArrere.java

Java

```
int comptador = 5;
do {
    System.out.println(comptador);
    comptador--;
} while (comptador ≥ 0);
System.out.println("Enlairament! 🚀");
```

5

4

3

2

1

0

Enlairament! 🚀

Mini-chequeig: bucles



Respon en 30 segons:

1. Quantes vegades s'executa el bloc d'un while si la condició és false des del principi?
2. I en un do-while?
3. Què és un bucle infinit i com s'ix d'ell en l'IDE?
4. Quan usaríes un do-while per a un menú?



Respostes:

- **Zero vegades:** el while comprova abans d'executar
- **Almenys una vegada:** el do-while executa i comprova després
- Un bucle la condició del qual mai no passa a false; es talla amb el botó  de l'IDE
- Quan vulgues mostrar el menú **sempre almenys una vegada** fins a triar opció vàlida

Resum del bloc 01



if / else if / else

El semàfor: guanya el primer true; ordena de més exigent a més laxa



switch

Valors concrets; sense break → fall-through; default és el comodí



while / do-while

Comprova abans vs després; la condició ha d'avançar cap a false



Bloc 02: el bucle **for**, els bucles anidats, **break** i **continue...** i les **excepcions** fan la seua entrada.

